

Universidade do Vale do Itajaí - Projeto de Sistemas Digitais
Professores: Douglas Rossi de Melo
Avaliação 1 – Projeto de Circuitos Combinacionais usando VHDL

Instruções:

1. Esta avaliação pode ser realizada em grupos de até três alunos.
2. Esta avaliação tem por objetivo consolidar o aprendizado sobre o projeto de circuitos combinacionais usando VHDL.
3. Os circuitos devem ser descritos em VHDL e verificados com o uso de testbenches no ambiente EDA Playground.
4. Aplique o seguinte guia de estilo na codificação VHDL:
 - a. Tabulação com 02 espaços (substituindo tabs por espaços);
 - b. Palavras reservadas da linguagem em caixa baixa;
 - c. Nomes de sinais em caixa alta (exceto os prefixos);
 - d. Utilize os seguintes prefixos para identificar o tipo sinal:
 - i_NOME : pino de entrada
 - o_NOME : pino de saída
 - w_NOME : fio interno
 - u_NOME : instância de componente
5. Deve ser postado um relatório básico no **Portfólio do Material Didático** que contenha:
 - a. Identificação do autor e do trabalho
 - b. Enunciado de cada circuito
 - c. Códigos VHDL dos circuitos e testbenches
 - d. Diagramas de formas de onda das simulações
 - e. Captura de tela do console indicando o resultado de cada simulação
 - f. Discussão dos resultados
6. A defesa da atividade será realiza na data lançada no sistema (presencial).

Circuitos:

Circuito 1. Apenas rode a simulação do circuito multiplexador 2-para-1 com canal de dados de 1 bit. O código do projeto e do seu testbench estão em anexo como referência para os demais projetos. **[1.0 ponto]**

Circuito 2. Implemente um circuito demultiplexador 1-para-2 com canal de dados de 1 bit e o seu testbench. **[3.0 pontos]**

Circuito 3. Implemente um circuito codificador binário de 2 bits para *one-hot* (<https://en.wikipedia.org/wiki/One-hot>) de 4 bits e o seu testbench. **[3.0 pontos]**

Circuito 4. Implemente um circuito decodificador *one-hot* de 4 bits para binário de 2 bits e o seu testbench. **[3.0 pontos]**

Códigos de referência

Circuito 1 - Design

```
library ieee;
use ieee.std_logic_1164.all;

entity mux2_1bit is
port ( i_SEL : in  std_logic;  -- selector
       i_A   : in  std_logic;  -- data input
       i_B   : in  std_logic;  -- data input
       o_S   : out std_logic); -- data output
end mux2_1bit;

architecture arch_1 of mux2_1bit is
begin
    process(i_SEL, i_A, i_B)
    begin
        if (i_SEL='0') then
            o_S <= i_A;
        else
            o_S <= i_B;
        end if;
    end process;
end arch_1;
```

Circuito 1 - Testbench

```
library ieee;
use ieee.std_logic_1164.all;

entity tb_mux2_1bit is
-- empty
end tb_mux2_1bit;

architecture arch_1 of tb_mux2_1bit is

-- DUT component
component mux2_1bit is
port ( i_SEL : in  std_logic;  -- selector
       i_A   : in  std_logic;  -- data input
       i_B   : in  std_logic;  -- data input
       o_S   : out std_logic); -- data output
end component;

signal w_SEL, w_A, w_B, w_S : std_logic;

begin

    -- Connect DUT
    u_DUT: mux2_1bit port map(i_SEL => w_SEL,
                             i_A   => w_A,
                             i_B   => w_B,
                             o_S   => w_S);

    process
    begin
        -- Select input channel A
        w_SEL    <= '0';
        w_A      <= '0';
        w_B      <= '0';
        wait for 1 ns;
        assert(w_S='0') report "Fail @ 000" severity error;
    end process;
```

```

w_SEL          <= '0';
w_A            <= '0';
w_B            <= '1';
wait for 1 ns;
assert(w_S='0') report "Fail @ 001" severity error;

w_SEL          <= '0';
w_A            <= '1';
w_B            <= '0';
wait for 1 ns;
assert(w_S='1') report "Fail @ 010" severity error;

w_SEL          <= '0';
w_A            <= '1';
w_B            <= '1';
wait for 1 ns;
assert(w_S='1') report "Fail @ 011" severity error;

-- Select input channel B
w_SEL          <= '1';
w_A            <= '0';
w_B            <= '0';
wait for 1 ns;
assert(w_S='0') report "Fail @ 100" severity error;

w_SEL          <= '1';
w_A            <= '0';
w_B            <= '1';
wait for 1 ns;
assert(w_S='1') report "Fail @ 101" severity error;

w_SEL          <= '1';
w_A            <= '1';
w_B            <= '0';
wait for 1 ns;
assert(w_S='0') report "Fail @ 110" severity error;

w_SEL          <= '1';
w_A            <= '1';
w_B            <= '1';
wait for 1 ns;
assert(w_S='1') report "Fail @ 111" severity error;

-- Clear inputs
w_SEL          <= '0';
w_A            <= '0';
w_B            <= '0';
assert false report "Test done." severity note;
wait;
end process;
end arch_1;

```